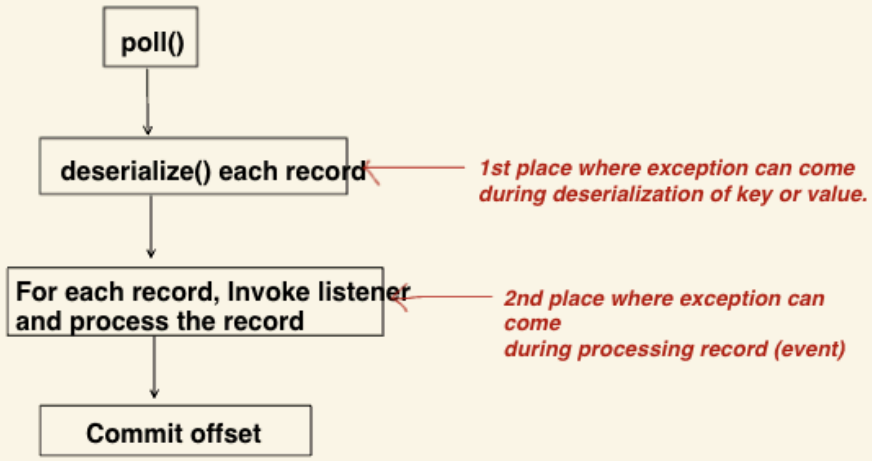
Exception handling in Kafka Consumer

Consumer process events

In consumer, there are 2 places during which exception can come:

1. exception comes during Deserialization?
2. exception comes during processing the record?

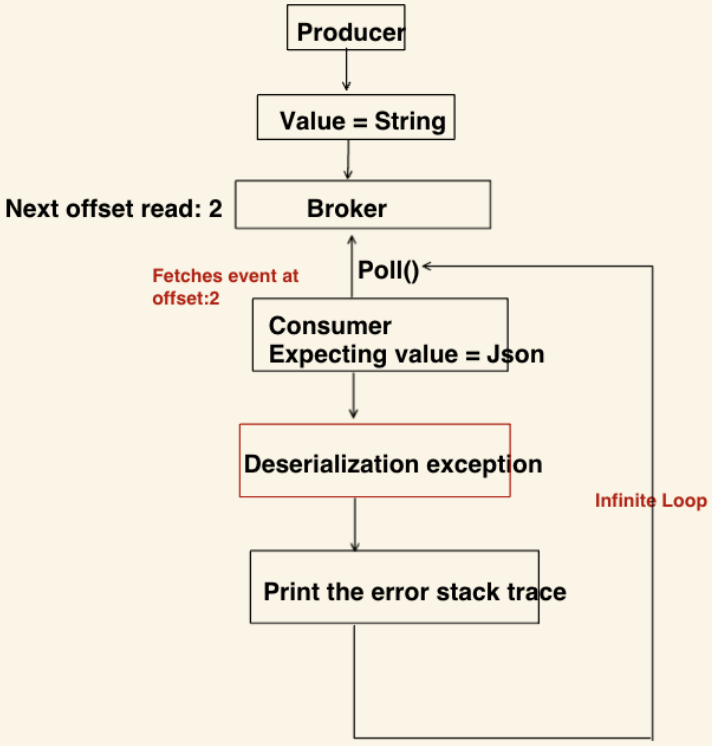
High level Flow:



Lets first see:

1st: Exception during Deserialization step

Problem: Infinite Loop



Even if we stop the consumer and restart, it will again start from offset : 2 and again stuck with the Infinite Loop

Logs in broker:

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log \
  --files /tmp/broker1-logs/order-events-0/00000000000000000000.log
Dumping /tmp/broker1-logs/order-events-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1000 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optiona
lLong.empty position: 0 CreateTime: 1774021256443 size: 245 magic: 2 compresscodec: snappy crc: 676649171 isvalid: true
| offset: 0 CreateTime: 1774021256443 keySize: 6 valueSize: 115 sequence: 0 headerKeys: [__TypeId__] key: O-2111 payload: {"orderId":"O-2111","customerId":"CUST-2","productId":"PROD-3","quant
ity":1,"totalAmount":99.99,"status":"CREATED"}
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: 1 lastSequence: 1 producerId: 1000 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optiona
lLong.empty position: 245 CreateTime: 1774021558338 size: 245 magic: 2 compresscodec: snappy crc: 1570887049 isvalid: true
| offset: 1 CreateTime: 1774021558338 keySize: 6 valueSize: 115 sequence: 1 headerKeys: [__TypeId__] key: O-2111 payload: {"orderId":"O-2111","customerId":"CUST-2","productId":"PROD-3","quant
ity":1,"totalAmount":99.99,"status":"CREATED"}
baseOffset: 2 lastOffset: 2 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1001 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optiona
lLong.empty position: 490 CreateTime: 1774022125204 size: 114 magic: 2 compresscodec: snappy crc: 139564955 isvalid: true
| offset: 2 CreateTime: 1774022125204 keySize: 6 valueSize: 18 sequence: 0 headerKeys: [] key: O-2111 payload: this is not a json
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Offset 2 is the buggy message:
Offset 0: value is json
Offset 1: value is json
Offset 2: value is String

Consumer Group offset status :

```
GROUP                TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                                  HOST
order-consumer-group order-events   0          2               3               1               consumer-order-consumer-group-1-d93bdb57-d81d-4360-a75a-29acf8bf8a3a /127.0.0.1
order-consumer-group order-events   1          0               0               0               consumer-order-consumer-group-1-d93bdb57-d81d-4360-a75a-29acf8bf8a3a /127.0.0.1
order-consumer-group order-events   2          0               0               0               consumer-order-consumer-group-2-1c7b3ce9-f442-4efe-a137-44f8ea804229 /127.0.0.1
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Next offset which consumer will read

Total number of offsets in this partition

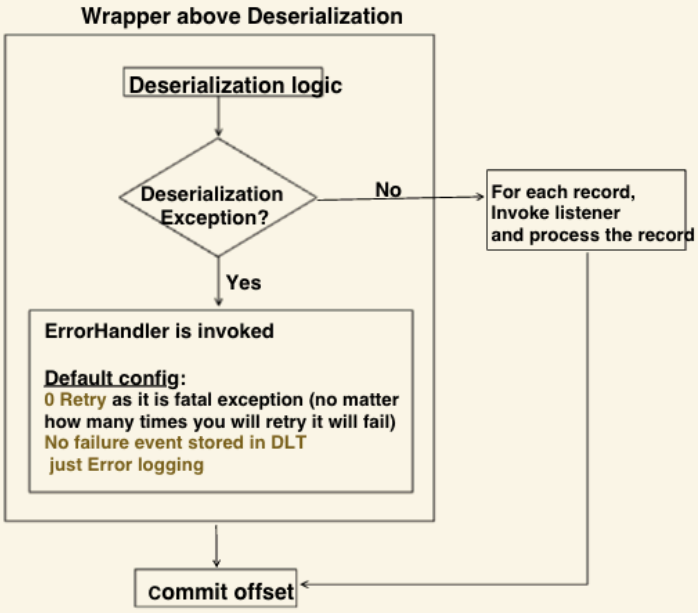
1 offset is yet to read.

Consumer log:

Not able to deserialize offset 2 event

```
    at org.springframework.kafka.listener.KafkaMessageListenerContainer$ListenerConsumer.run(KafkaMessageListenerContainer.java:1387) ~[spring-kafka-3.3.10.jar:3.3.10]
    at java.base/java.util.concurrent.CompletableFuture$AsyncRun.run(CompletableFuture.java:1804) ~[na:na] <1 internal line>
Caused by: org.apache.kafka.common.errors.RecordDeserializationException Create breakpoint : Error deserializing VALUE for partition order-events-0 at offset 2. If needed, please seek past the record
    at org.apache.kafka.clients.consumer.internals.CompletedFetch.newRecordDeserializationException(CompletedFetch.java:346) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.CompletedFetch.parseRecord(CompletedFetch.java:330) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.CompletedFetch.fetchRecords(CompletedFetch.java:286) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.FetchCollector.fetchRecords(FetchCollector.java:169) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.FetchCollector.collectFetch(FetchCollector.java:135) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.Fetcher.collectFetch(Fetcher.java:148) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.ClassicKafkaConsumer.pollForFetches(ClassicKafkaConsumer.java:672) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.ClassicKafkaConsumer.poll(ClassicKafkaConsumer.java:623) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.ClassicKafkaConsumer.poll(ClassicKafkaConsumer.java:596) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.KafkaConsumer.poll(KafkaConsumer.java:876) ~[kafka-clients-3.9.1.jar:na]
    at org.springframework.kafka.listener.KafkaMessageListenerContainer$ListenerConsumer.pollConsumer(KafkaMessageListenerContainer.java:1692) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.springframework.kafka.listener.KafkaMessageListenerContainer$ListenerConsumer.doPoll(KafkaMessageListenerContainer.java:1667) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.springframework.kafka.listener.KafkaMessageListenerContainer$ListenerConsumer.pollAndInvoke(KafkaMessageListenerContainer.java:1445) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.springframework.kafka.listener.KafkaMessageListenerContainer$ListenerConsumer.run(KafkaMessageListenerContainer.java:1335) ~[spring-kafka-3.3.10.jar:3.3.10]
    ... 2 common frames omitted
Caused by: org.apache.kafka.common.errors.SerializationException Create breakpoint : Can't deserialize data  from topic [order-events]
    at org.springframework.kafka.support.serializer.JsonDeserializer.deserialize(JsonDeserializer.java:603) ~[spring-kafka-3.3.10.jar:3.3.10]
    at org.apache.kafka.common.serialization.Deserializer.deserialize(Deserializer.java:81) ~[kafka-clients-3.9.1.jar:na]
    at org.apache.kafka.clients.consumer.internals.CompletedFetch.parseRecord(CompletedFetch.java:327) ~[kafka-clients-3.9.1.jar:na]
    ... 14 common frames omitted
Caused by: com.fasterxml.jackson.core.JsonParseException: Unrecognized token 'this': was expecting (JSON String, Number, Array, Object or token 'null', 'true' or 'false')
 at [Source: REDACTED (`StreamReadFeature.INCLUDE_SOURCE_IN_LOCATION` disabled); line: 1, column: 6]
    at com.fasterxml.jackson.core.JsonParser._constructError(JsonParser.java:2596) ~[jackson-core-2.19.2.jar:2.19.2]
    at com.fasterxml.jackson.core.JsonParser._constructReadException(JsonParser.java:2622) ~[jackson-core-2.19.2.jar:2.19.2]
    at com.fasterxml.jackson.core.JsonParser._constructReadException(JsonParser.java:2630) ~[jackson-core-2.19.2.jar:2.19.2]
    at com.fasterxml.jackson.core.base.ParserMinimalBase._reportError(ParserMinimalBase.java:830) ~[jackson-core-2.19.2.jar:2.19.2]
    at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._reportInvalidToken(UTF8StreamJsonParser.java:3662) ~[jackson-core-2.19.2.jar:2.19.2]
    at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._reportInvalidToken(UTF8StreamJsonParser.java:3635) ~[jackson-core-2.19.2.jar:2.19.2]
```

Solution: Error Handling



Lets first add the wrapper:

Consumer: application.properties

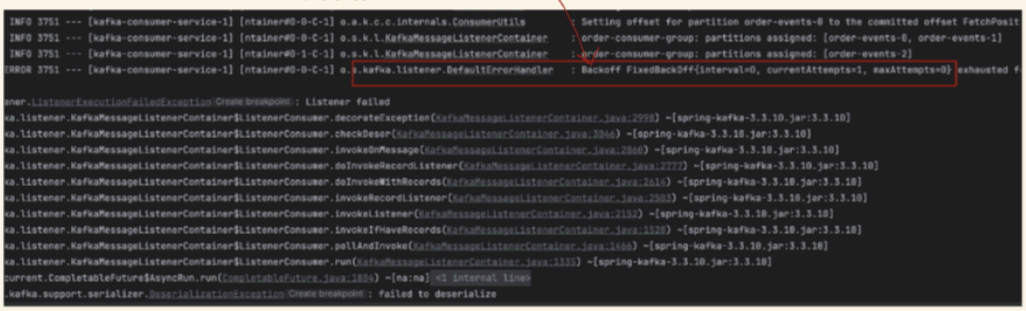
```
#Use wrapper
spring.kafka.consumer.key-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer

#wrapper above actual class
spring.kafka.consumer.properties.spring.deserializer.key.delegate.class=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.properties.spring.deserializer.value.delegate.class=org.springframework.kafka.support.serializer.JsonDeserializer
```

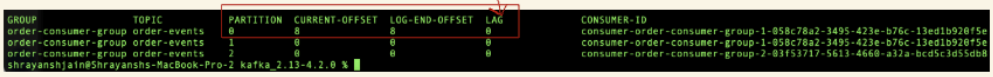
At this point we are not adding Error Handler, and relying on default error handler:

That's it, wrapper and all is now handled by the framework.

DefaultErrorHandler got invoked, and in this for deserialization error, retry strategy used is FixedBackOff with maxAttempts = 0, means do not retry if its fails once.



No lag, means consumer committed the offset too for which deserialization issue occurs.



But we lost the event (record). But in production we should not rely on DefaultErrorHandler.

Instead, we should have our Error Handler where we store the failed event in DLT.

So that after fix, it can be replayed or say some action can be taken on those.

<u>Now lets create Error handler Bean:</u>

Different Retry Strategies:

1. FixedBackOff

```
@Bean
public DefaultErrorHandler errorHandler() {
    DefaultErrorHandler handler = new DefaultErrorHandler(
                    new FixedBackOff(1000L, 2));
    return handler;
}
```

Wait for 1000ms (1sec) before retrying again. And max 2 retries need to be done.

Attempt1: fail
Wait 1 sec
Attempt2 (Retry-1) : fail
Wait 1 sec
Attempt3 (Retry-2) : fail

2. ExponentialBackOffWithMaxRetries

```
@Bean
public DefaultErrorHandler errorHandler() {

    ExponentialBackOffWithMaxRetries backOff = new
    ExponentialBackOffWithMaxRetries(5); backOff.setInitialInterval(1000L);
    backOff.setMultiplier(2.0);
    backOff.setMaxInterval(10000L);

    DefaultErrorHandler handler = new DefaultErrorHandler(backOff );
    return handler;
}
```

First retry happens after 1s delay
*Each subsequent retry double the wait time (*2)*

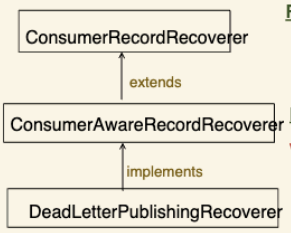
- Retry 1: 1s wait
- Retry 2: 2s wait
- Retry 3: 4s wait
- Retry 4: 8s wait

Max delay can be 10sec (if after multiplying delays > 10s, use 10s only)

And max retry can be 5 only.

Retry feature is added in Error Handler but still if all the Retries finished, what to do?

That's where Recoverer comes into the picture:



Functional Interface:

```
void accept(T t, U u);
```

Functional Interface:

```
void accept(ConsumerRecord<?, ?> record, Exception exception)
```

Class:

```
void accept(ConsumerRecord<?, ?> record, Exception exception)
{
        // logic to insert record in DLT
}
```

Here if we see, 1 implementation "*DeadLetterPublishingRecoverer*" is already provided by framework.

And this class "accept" method take care of inserting the failed event in DLT.

But

If we want our own recoverer, we can do that too:
- Like if we want to insert the failed record in DB instead of DLT.
- Like if we want to send alert

etc.

Will see that.

We will use below override method of ErrorHandler to set both Recoverer and Retry

```
public DefaultErrorHandler(ConsumerRecordRecoverer recoverer, BackOff backOff) {
        ...
}
```

Lets add DLT feature in Error Handler:

Created DLT, in which failed events will be stored

```
@Bean
public NewTopic orderDltEventsTopic() {
        return TopicBuilder.name("order-events-dlt").build();
}
@Bean
public DeadLetterPublishingRecoverer deadLetterPublishingRecoverer(KafkaTemplate<Object, Object>
template) {
        return new DeadLetterPublishingRecoverer(template);
}
@Bean
public DefaultErrorHandler errorHandler(DeadLetterPublishingRecoverer recoverer) {
        DefaultErrorHandler handler = new DefaultErrorHandler(recoverer,
                        new FixedBackOff(1000L, 2));
        return handler;
}
```

Created a Bean of DeadLetterPublishingRecoverer, this framework class has all the code to push records for which deserialization fails to DLT.

All we need to provide is KafkaTemplate object.

KafkaTemplate object has properties created from application.properties filed

Like key serilizer, value serilizer etc.

Now, Error handler has both Retry feature + DLT

So after all retry finished.
Error handler will invoke ***DeadLetterPublishingRecoverer* "*accept*"**
method and pass the ConsumerRecord and Exception object.

NOTE (BRAINSTORMING PART):

Usecase1 : value deserilization failed

ConsumerRecord :
Value = null
Key = O-111
Header:
- springDeserializerExceptionValue : value bytes[]

```
ProducerRecord<Object, Object> dltRecord = new ProducerRecord<>(
    "order-events-dlt",
    "O-111",
     valueBytes
);
```



```
#----------------SERIALIZER which we need to add -------------
# for Key
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
#for value
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.ByteArraySerializer
```

Usecase2 : key deserilization failed

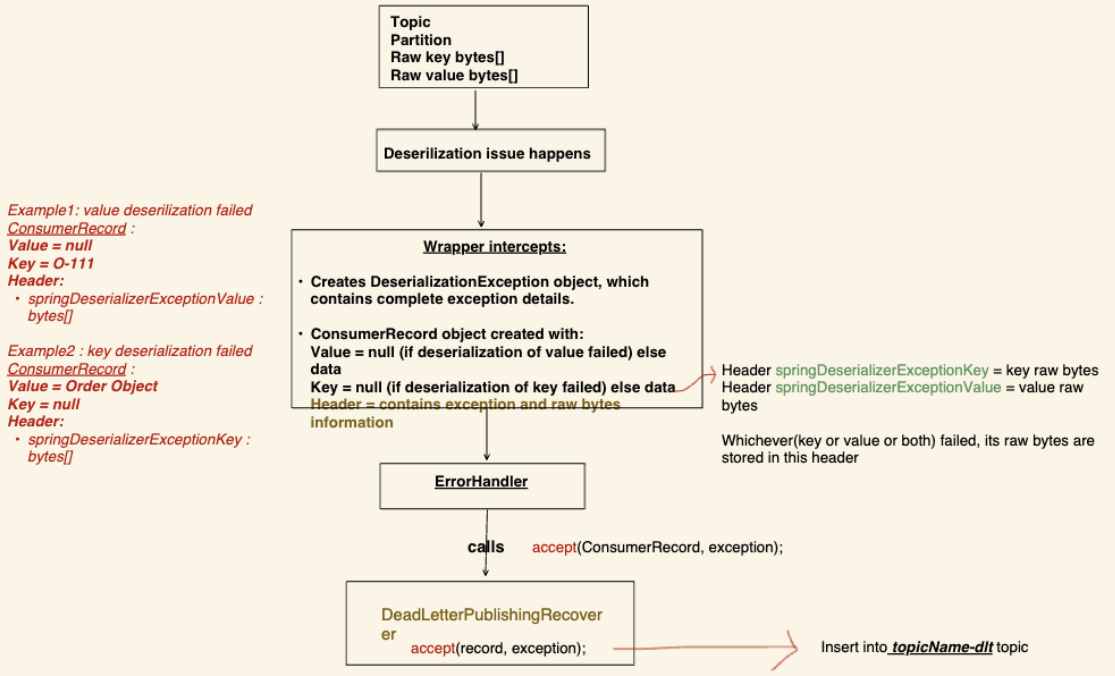
ConsumerRecord :
Value = Order Object
Key = null
Header:
- springDeserializerExceptionKey : key bytes[]

```
ProducerRecord<Object, Object> dltRecord = new ProducerRecord<>(
    "order-events-dlt",
    keyBytes,
     Order
);
```

```
#----------------SERIALIZER which we need to add -------------
# for Key
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.ByteArraySerializer
#for value
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer
```

But that's challenging right.

But how this DLT works? What is happening internally?



Conceptual internal working of accept method:

```
public void accept(ConsumerRecord<?, ?> record, Exception exception) {

    // 1. Find the raw  bytes from the exception
    byte[] rawKeyBytes = ((DeserializationExceptionexception).getHeader("springDeserializerExceptionKey");
    byte[] rawValueBytes = ((DeserializationExceptionexception).getHeader("springDeserializerExceptionValue");

   // 2. Create a NEW record for the DLT topic
    ProducerRecord<Object, Object> dltRecord = new ProducerRecord<>(
        record.getTopic()-"dlt",  // destination topic
        record.key() != null ? record.key() :  rawKeyBytes ,    // key could be null, if deserialization issue with key, then we add raw bytes
         record.value() != null ? record.value() :  rawValueBytes);   // value could be null, if deserialization issue with Value, then we add raw
bytes

    //3. send it
    kafkaTemplate.send(dltRecord);
}
```

But that's challenging right.

My take:

Most of the scenario, key is String, so very less chance of deserialization issue with key.

Mostly deserilization issue comes with value (Json serilization and deserilization). And when deserialization issue comes with value, raw bytes[] are going to be published.

So below sterilization should work fine:

```
# for Key
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
#for value
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.ByteArraySerializer
```

But if there is a scenario that your Key is also an Object. Then deserialization issue could be with:

- Only key
- Only value
- Both key and value

Then its challenging whether to use ByteArraySerializer or JsonSerializer

My answer would be, Override the produceRecord method and always make sure both key and value are in bytes only.

and then use ByteSerializer for both:

```
# for Key
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.ByteArraySerializer
#for value
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.ByteArraySerializer
```

Just overrided this method and covert key and value to bytes

```
@Bean
public DeadLetterPublishingRecoverer deadLetterPublishingRecoverer(KafkaTemplate<Object, Object> template) {

        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template) {

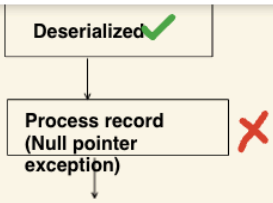
                @Override
                protected ProducerRecord<Object, Object> createProducerRecord(ConsumerRecord<?, ?> record,
                                                TopicPartition tp, Headers headers, byte[] key, byte[] value) {

                        Object finalKey = (key != null) ? key : objectMapper.writeValueAsBytes(record.key()));
                        Object finalValue = (value != null) ? value : objectMapper.writeValueAsBytes(record.value());

                        return new ProducerRecord<>(tp.topic(), tp.partition(), finalKey, finalValue, headers);
                }
        };
        return recoverer;
}

@Bean
public DefaultErrorHandler errorHandler(DeadLetterPublishingRecoverer recoverer) {

        DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 2));
        return handler;
}
```



- Default Error Handler catch the exception:
  - Log it
  - Commit the offset

Note:
- For deserialization issues, we have to wrap the error, so that Default Error Handler is provided.
- But for exception during processing, default error handler is provided by default.

Now, if we want Retry + after all retry finished then retryer feature (like DLT) is everything same what we have done for deserialization.

Custom Retryer in Error Handler:

```
@Bean
public DefaultErrorHandler errorHandler(DeadLetterPublishingRecoverer recoverer) {
        DefaultErrorHandler handler = new DefaultErrorHandler(
                        (record, exception) -> {
                                System.out.println("Failed record: " + record);
                        } ,
                        new FixedBackOff(1000L, 2));
        return handler;
}
```

```
2026-03-21T12:46:01.543+05:30  INFO 12685 --- [kafka-consumer-service-1] [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerUtils      : Setting offset for partition order-events-1 to the committed offset FetchPosition{offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9192 (id: 4 rack: null)], epoch=0}}
2026-03-21T12:46:01.543+05:30  INFO 12685 --- [kafka-consumer-service-1] [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerUtils      : Setting offset for partition order-events-0 to the committed offset FetchPosition{offset=16, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:9092 (id: 3 rack: null)], epoch=0}}
2026-03-21T12:46:01.544+05:30  INFO 12685 --- [kafka-consumer-service-1] [ntainer#0-0-C-1] o.s.k.l.KafkaMessageListenerContainer    : order-consumer-group: partitions assigned: [order-events-0, order-events-1]
2026-03-21T12:46:01.544+05:30  INFO 12685 --- [kafka-consumer-service-1] [ntainer#0-1-C-1] o.s.k.l.KafkaMessageListenerContainer    : order-consumer-group: partitions assigned: [order-events-2]
Failed record: ConsumerRecord(topic = order-events, partition = 0, leaderEpoch = 0, offset = 16, CreateTime = 1774077360135, serialized key size = 5, serialized value size = 117, headers = RecordHeaders(headers = [RecordHeader(key = springDeserializerExceptionKey, value = [-84, -19, 0, 5, 115, 114, 0, 69, 111, 114, 103, 46, 115, 112, 114, 105, 110, 103, 102, 114, 97, 109, 101, 119, 111, 114, 107, 46, 107, 97, 102, 107, 97, 46, 115, 117, 112, 112, 111, 114, 116, 46, 115, 101, 114, 105, 97, 108, 105, 122, 101, 114, 46, 68, 101, 115, 101, 114, 105, 97, 108, 105, 122, 97, 116, 105, 111, 110, 69, 120, 99, 101, 112, 116, 105, 111, 110, 114, -24, -116, 126, -45, 78, 67, -115, 2, 0, 2, 90, 0, 5,46, 108, 97, 110, 103, 46, 84, 104, 114, 111, 119, 97, 98, 108, 101, -43, -58, 53, 39, 57, 119, -72, -53, 3, 0, 4, 76, 0, 5, 99, 97, 117, 115, 101, 116, 0, 21, 76, 106, 97, 118, 97, 47, 108, 97, 110, 103, 47, 84, 104, 114, 111, 119, 97, 98, 108, 101, 59, 120, 112, 113, 0, 126, 0, 5, 116, 0, 21, 76, 106, 97, 118, 97, 47, 108, 97, 110, 103, 47, 84, 104, 114, 111, 119, 97, 98, 108, 101, -43, -58, 53, 39, 57, 119, -72, -53, 3, 0, 4, 76, 0, 5, 99, 97, 117, 115, 101, 116, 0, 21, 76, 106, 97, 118, 97, 47, 108, 97, 110, 103, 47, 84, 104, 114, 111, 119, 97, 98, 108, 101, 59, 120, 112, 10 34])], isReadOnly = false), key = null, value = Order{orderId='Z-010101', customerId='CUST-2', productId='PROD-3', quantity=1, totalAmount=99.99, status='CREATED'}
```

Doubt: What if exception comes during processing the record?

